What the AI Transit Pipeline checks, file type by file type

34 scanner slides covering every extension the pipeline routes, and one on unknown extensions.

Each slide: the extensions · the tools that run · the rules that FAIL · the rules that only WARN · the known limits.

Source of truth: classify_file() in scan_pipeline.sh. python3 build_pdf_filetypes.py --check fails if this deck drifts from it.

What every file gets

Layer 5 is the part that varies by type. The other layers do not look at the extension.

L1	**Global**	Whole directory	gitleaks · detect-secrets · ClamAV · YARA
L2	**Semgrep**	Whole directory	p/owasp-top-ten · p/cwe-top-25 · p/security-audit · p/secrets
L3	**Dependencies**	Manifests	trivy · pip-audit · safety · npm audit
L4	**Universal patterns**	EVERY file	Credentials · 18 known token formats · PEM keys · traversal · SSRF · weak crypto / PRNG
L5	**Per-type scanner**	By type	One slide per type in this deck
L6	**ScanCode**	Whole directory	Licences (GPL / AGPL / LGPL / SSPL / BUSL → WARN) · CVE HIGH / CRITICAL → FAIL

WARN = logged, the pipeline continues. FAIL = the repository is quarantined and no ZIP is produced. .transit-allow.json downgrades a named FAIL to WARN; .transitignore removes a file from every scan.

How a file is routed

classify_file() decides in this order; the first match wins.

0 **Excluded?**

.transitignore patterns and the default pruned directories (node_modules, .git, vendor, dist, build, target, coverage …) never reach a scanner.

1 **Basename**

Dockerfile*, Makefile*, CMakeLists.txt, package.json, composer.json, requirements*.txt, constraints*.txt, *.gradle, *.gradle.kts, gradle*.properties, setup.py, conanfile.py.

2 **Extension**

Lower-cased first, so .PS1 and .JAR are treated like .ps1 and .jar. One slide per family in this deck. Office extensions are only a hint: the content is sniffed (a .dot may be Graphviz).

3 **Shebang**

No known extension: the first line decides — python, bash / sh / zsh / ksh, ruby, node, php, pwsh / powershell.

4 **MIME type**

Still nothing: file --mime-type. An executable type (x-executable, x-sharedlib, x-dosexec) → FAIL.

5 **Default**

Anything else passes Layer 5 untouched. See "Unknown extensions".

Python

`scan_python`

`.py`

also by shebang: #!...python

TOOLS APPLIED

Bandit

bandit -q -l -i. Any MEDIUM or HIGH issue → FAIL (bandit:<severity>:<test id>). Not installed → WARN.

FAIL — blocks the repository

- CWE-78 os.system, subprocess.call, subprocess.Popen, commands.getoutput
- CWE-95 eval(or exec(
- CWE-502 pickle.load(s), marshal.loads, yaml.load(without a Loader
- CWE-89 execute / executemany with an f-string, +, % or .format() query
- CWE-703 assert is_admin / is_authenticated / has_perm / user.is...

WARN — reported, does not block

- CWE-601 redirect(or HttpResponseRedirect(fed from request....
- bandit missing

Note: Broad by design: eval, exec and os.system fail even with a literal argument. setup.py and conanfile.py are routed to the install-time scanner instead.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Install-time Python — setup.py, conanfile.py

`scan_setup_py`

`setup.py` `conanfile.py`

TOOLS APPLIED

— none —

Replaces scan_python for these two names: Bandit and the generic eval / exec / os.system FAILs do not run on them.

FAIL — blocks the repository

- A download piped into a shell
- exec / eval of a network response
- A blob decoded and executed in the same file (whole-file test)
- A socket handed to a shell, or a pty
- Writing authorized_keys, a crontab or a systemd unit

WARN — reported, does not block

- Overriding install / develop / egg_info
- Downloading at install time
- Running a program

Note: setup.py runs on a source pip install and a Conan recipe on conan install, so both fire before anything is imported. 36 real setup.py: 4 FAIL → 0 FAIL, 3 WARN.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

JavaScript / TypeScript

`scan_javascript`

`.js` `.ts` `.jsx` `.tsx` `.mjs` `.cjs`

also by shebang: #!...node

TOOLS APPLIED

Semgrep p/javascript

Run per file. Any finding → FAIL (semgrep:js:N_findings). Not installed → WARN.

FAIL — blocks the repository

- CWE-79 innerHTML / outerHTML / document.write / insertAdjacentHTML assignment
- CWE-78 child_process, exec(, execSync(, spawn(
- CWE-95 eval(or new Function(
- CWE-1321 __proto__, constructor[prototype], Object.assign({}
- CWE-915 Object.assign / _.merge / deepmerge(with req.... as the source
- CWE-89 SQL in a template literal (query / raw / knex.raw / sequelize.query)

WARN — reported, does not block

- CWE-611 DOMParser, parseXML, libxmljs.parseXml
- semgrep missing

Note: package.json is routed to the manifest scanner, not here.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Java source

scan_java

.java

TOOLS APPLIED

— none per file —

Semgrep covers Java at Layer 2 (whole directory).

FAIL — blocks the repository

- CWE-78 Runtime.getRuntime().exec(or ProcessBuilder(
- CWE-89 createQuery / createNativeQuery / prepareStatement with "…" +
- CWE-502 ObjectInputStream, XMLDecoder, XStream
- CVE-2021-44228 jndi:, ldap://, rmi://, ${jndi (Log4Shell)
- CWE-295 TrustAllCerts / an empty X509TrustManager

WARN — reported, does not block

- CWE-79 request.getParameter written to the response
- CWE-611 DocumentBuilderFactory.newInstance() — check the entity configuration

Note: Compiled Java is a different file type: .jar .war .ear .aar have their own scanner.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

PHP

scan_php

.php

also by shebang: #!...php

TOOLS APPLIED

— none per file —

Semgrep covers PHP at Layer 2 (whole directory).

FAIL — blocks the repository

- CWE-78 shell_exec, exec, system, passthru, popen, proc_open
- CWE-89 mysql_query / mysqli_query / pg_query concatenating a variable
- CWE-79 echo $_GET / $_POST / $_REQUEST / $_COOKIE / $_SERVER
- CWE-22 include / require with a variable path
- CWE-502 unserialize(of $_GET / $_POST / $_COOKIE / $_REQUEST

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: Every rule here is a FAIL: PHP has no reviewable tier.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Ruby

`scan_ruby`

`.rb`

also by shebang: #!...ruby

TOOLS APPLIED

— none per file —

Semgrep covers Ruby at Layer 2 (whole directory).

FAIL — blocks the repository

- CWE-78 system, exec, Kernel.exec, %x{ or a backtick command
- CWE-95 eval(/ eval {
- CWE-89 where / find_by_sql / execute with "...#{...}" interpolation
- CWE-22 File.read / File.open / IO.read of params[...]

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: A Rakefile is not routed to this scanner: it was measured and rejected (8 of 62 false positives on markdown backticks).

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Go

scan_go

.go

TOOLS APPLIED

— none per file —

Semgrep covers Go at Layer 2 (whole directory).

FAIL — blocks the repository

- CWE-89 fmt.Sprintf building SELECT / INSERT / UPDATE / DELETE

WARN — reported, does not block

- CWE-78 exec.Command(
- CWE-22 os.Open / ioutil.ReadFile / http.ServeFile of r.URL or path
- CWE-338 "math/rand" imported
- CWE-326 tls.Config{ without MinVersion

Note: Mostly reviewable-tier: only SQL built by fmt.Sprintf blocks.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

C / C++

`scan_c_cpp`

`.c` `.cpp` `.cc` `.h` `.hpp`

TOOLS APPLIED

cppcheck

--enable=all. Any error: or warning: line → FAIL (cppcheck:<first>). Not installed → WARN.

FAIL — blocks the repository

- CWE-120 gets, gets_s, strcpy, strcat, sprintf, vsprintf
- CWE-78 system, popen, execl / execle / execlp / execv / execve / execvp
- CWE-134 printf-family call with argv, getenv, user_input or stdin as the format
- CWE-377 tmpnam, tempnam, mktemp (TOCTOU temp file)

WARN — reported, does not block

- CWE-190 malloc(a * b) — possible integer overflow
- CWE-415 two free() calls on one line — possible double free
- CWE-338 rand()

Note: CERT STR31 / MEM30 / INT30 / ENV33 / FIO45 / MSC30 are the reference points.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Rust

`scan_rust`

`.rs`

TOOLS APPLIED

— none —

Three built-in rules, deliberately small.

FAIL — blocks the repository

- CWE-119 unsafe { block — needs human review
- CWE-78 std::process::Command
- CWE-798 password / secret / api_key / token assigned a 4+ character literal

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: Known gap: a build.rs legitimately uses std::process::Command (how pkg-config and cc run), so it over-fires. No build.rs existed on the host to calibrate a better rule.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Kotlin

`scan_kotlin`

`.kt` `.kts`

TOOLS APPLIED

— none —

*.gradle.kts is a Gradle build script, not Kotlin source: it goes to the Gradle scanner.

FAIL — blocks the repository

- CWE-78 Runtime.getRuntime().exec(
- CWE-798 password / secret / apiKey / api_key / token assigned a 4+ character literal
- CWE-327 MessageDigest.getInstance("MD5") or "SHA-1"

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: foo.main.kts and other non-Gradle Kotlin scripts still come here.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

C#

`scan_csharp`

`.cs`

TOOLS APPLIED

— none —
Four built-in rules.

FAIL — blocks the repository

- CWE-78 Process.Start(
- CWE-798 password / secret / apiKey / api_key / token assigned a 4+ character literal
- CWE-327 new MD5CryptoServiceProvider()
- CWE-89 a "SELECT …" literal followed by + (concatenation)

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: Only SELECT is matched for SQL concatenation.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Shell

`scan_shell`

`.sh` `.bash` `.zsh` `.ksh`

also by shebang: #!.../bash #!.../sh #!.../zsh #!.../ksh #!/usr/bin/env bash|sh

TOOLS APPLIED

ShellCheck
shellcheck -S warning. Any warning → FAIL (shellcheck:warnings). Not installed → WARN.

FAIL — blocks the repository

- CWE-78 curl ... | bash / sh and wget ... | bash / sh
- CWE-88 eval $variable

WARN — reported, does not block

- CWE-426 export PATH= starting with a relative directory
- CWE-377 predictable temp name (/tmp/$$, /tmp/<name>tmp)
- IFS manipulation
- shellcheck missing

Note: Makefile recipes, PowerShell and batch files have their own scanners.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

PowerShell

`scan_powershell · scan_powershell_xml`

`.ps1` `.psm1` `.psd1` `.pssc` `.psrc` `.ps1xml` `.psc1` `.cdxml`

also by shebang: #!…pwsh #!…powershell

TOOLS APPLIED

— none —

One shared rule set (_ps_rules). The XML types (.ps1xml .psc1 .cdxml) also get scan_xml's external-entity check.

FAIL — blocks the repository

- Fetch then execute: irm … | iex, DownloadString + iex, or a download piped to a shell
- -EncodedCommand and its abbreviations (-enc, -e) followed by a base64 blob
- FromBase64String with iex / ScriptBlock::Create / Assembly::Load / VirtualAlloc (whole file)
- A backtick inside the command name, an AMSI bypass, Defender disabled
- Persistence: Run key, WMI event consumer, Startup folder; socket fed to a shell; VirtualAlloc + CreateThread
- A literal $Password, or ConvertTo-SecureString "…" -AsPlainText
- A plaintext http:// download that is then run

WARN — reported, does not block

- Invoke-Expression alone (a deliberate relaxation)
- Start-Process / cmd /c, an https download then run
- Execution-policy bypass, hidden window, Defender exclusion
- Scheduled task, certificate validation off, local account creation
- ScriptsToProcess in a manifest; wildcard VisibleCmdlets in a role capability

Note: Matching is case-insensitive, and so is the extension (UPPER.PS1 used to bypass the scanner). 32 real scripts: 8 FAIL (all false positives) → 0. None of the module, manifest or XML types exists on the host, so those routes rest on fixtures. Legacy .msh is not routed.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Windows batch

`scan_bat`

`.bat` `.cmd`

TOOLS APPLIED

— none —

Works on a normalised view: CR stripped, REM / :: comment lines dropped, ^ continuations joined. Case-insensitive.

FAIL — blocks the repository

- certutil -urlcache, bitsadmin /transfer, a PowerShell DownloadString cradle, curl -k
- powershell -enc <base64> (any abbreviation); certutil -decode into an executable
- Download then execute on one line; carets inside a command name
- A Run / RunOnce / Winlogon / IFEO key, or a file copied into the Startup folder
- UAC, Defender or the firewall turned off; a new member of local Administrators
- A delete aimed exactly at a drive root or the profile; format; set …PASSWORD=<literal>

WARN — reported, does not block

- net user /add, schtasks /create, certutil -decode of anything else
- Fetch and run in one script; a plaintext http:// download
- -ExecutionPolicy Bypass, hidden PowerShell window
- mshta / wscript / rundll32 / regsvr32; wmic process call create; a remote UNC path
- A command assembled from adjacent %v:~n,1% extractions

Note: 116 real files, all clean — but they are npm shims, virtualenv and Maven wrappers, with no Windows-native automation, so that says little about recall. .btm and the 4DOS family are not routed.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Makefile

`scan_makefile`

`.mk` `.mak` `.make` `Makefile*` `makefile*` `GNUmakefile*`

TOOLS APPLIED

— none —

$(shell …) runs when make merely reads the file, before any target; recipe lines are shell.

FAIL — blocks the repository

- A download piped into a shell; a parse-time fetch $(shell curl …)
- A base64-decoded payload executed; a reverse shell
- A hardcoded credential (assignments are unquoted, so the universal rule cannot see them)
- TLS verification disabled
- A setuid / setgid bit; cron or authorized_keys persistence
- rm -rf of exactly /, ~ or $HOME

WARN — reported, does not block

- sudo in a recipe
- chmod 777
- A plaintext http:// download
- Clone-and-build from a remote

Note: 88 real Makefiles: no finding of any kind. No rule for a bare $(eval …): it fired on 7 of 88 as ordinary metaprogramming.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

CMake

`scan_cmake`

`.cmake` `CMakeLists.txt` `CMakeLists.*.txt`

TOOLS APPLIED

— none —

file(DOWNLOAD) and execute_process() run at configure time: cmake . is enough.

FAIL — blocks the repository

- A credential in set(NAME "value") — a function call the universal rule cannot see
- A plaintext http:// download, or TLS_VERIFY OFF
- A download piped into a shell
- Hardening removed: -fno-stack-protector, _FORTIFY_SOURCE=0, execstack
- file(WRITE …) into /etc, ~/.ssh or cron

WARN — reported, does not block

- A build step that fetches
- COMMAND sh -c
- install(CODE | SCRIPT)
- GIT_TAG on a moving branch

Note: 1 of 1116 real files fails: a plaintext archive URL in CMake's own ExternalProject documentation. execute_process appears in 106 of 1116, so it is not a finding by itself.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Gradle

`scan_gradle`

`gradle.properties` `gradle-wrapper.properties` `*.gradle` `*.gradle.kts`

TOOLS APPLIED

— none —

A Gradle script is a program, evaluated in full at configuration time (gradle tasks, --dry-run, an IDE import). settings.gradle runs first.

FAIL — blocks the repository

- Build scripts: apply from 'http(s)://...', allowInsecureProtocol = true
- Build scripts: a process started from a literal or list, a ProcessBuilder (matched without new, so Kotlin is covered), Runtime.exec
- gradle.properties: a literal *.password / *.secret / *.apiKey
- gradle.properties: dependency verification off or lenient; -javaagent / -agentpath in org.gradle.jvmargs
- gradle-wrapper.properties: a plaintext distributionUrl

WARN — reported, does not block

- A configuration-time remote fetch
- exec / commandLine / javaexec
- A trustStore override
- validateDistributionUrl=false

Note: 6413 real files, 99.6% pass; the 26 FAILs are credentials in Gradle's own docs and tests, one doc snippet, two genuine spawns. A missing distributionSha256Sum is deliberately not reported (20 of 26 omit it). gradle-wrapper.jar is a jar: see Java archives.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Package manifests

`scan_package_json · scan_requirements`

`package.json` `composer.json` | `requirements*.txt` `constraints*.txt`

TOOLS APPLIED

trivy · pip-audit · npm audit

At Layer 3, whole directory: CVEs in the dependencies a manifest names. This slide is about what the manifest itself can run.

FAIL — blocks the repository

- npm: preinstall / install / postinstall (and prepare) that pipe a download into a shell, decode base64, open /dev/tcp or use nc -e
- npm: a dependency on http://, git:// or git+http://
- Composer: the same hook rules; secure-http false; an http:// repository
- pip: an index, -f or requirement over http:// or git://; credentials embedded in a URL

WARN — reported, does not block

- An install-time hook being present
- A non-lifecycle script that pipes or fetches
- A git or tarball dependency
- --trusted-host; --extra-index-url (dependency confusion)
- A VCS requirement not pinned to a commit

Note: JSON is parsed, not grepped: "post\u0069nstall" is postinstall to npm. Calibration is weak: 0 of 1193 real package.json fail, but they are mostly published packages. Lockfiles, go.mod and meson_options.txt are not covered.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

XML, Maven and Ant

`scan_xml`

`.xml` `.pom`

TOOLS APPLIED

— none —

Matches on a newline-folded, comment-stripped view. pom.xml and build.xml get Maven / Ant rules by name.

FAIL — blocks the repository

- XXE: <!ENTITY … SYSTEM pointing at file:, expect:, php:, gopher:, jar:, netdoc:, ftp:, http(s): or an absolute path
- An entity expansion bomb: 4+ named references in one entity value
- A credential in <password> / <token> / <secret> / <apiKey> / <privateKey> with a 6+ character literal
- Maven: a <repository> whose <url> is http://; exec-maven-plugin / antrun running sh, bash, cmd, powershell, curl, wget
- Ant: <get> over http://; <exec> of a shell or downloader; <sshexec> / <scp> with a literal password=

WARN — reported, does not block

- A relative SYSTEM include (DocBook) or a PUBLIC DTD reference
- The exec or antrun plugin being present; system scope
- Any Ant <exec>

Note: A bare <!DOCTYPE failed 736 of 3362 real XML files, so it is no longer a rule: 0 of 3362 fail now. The Ant half rests on only 22 real files. No -SNAPSHOT rule (176 of 221 real poms use one).

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

YAML / CI / Kubernetes

`scan_yaml`

`.yml` `.yaml`

TOOLS APPLIED

— none per file —

checkov covers Terraform, not YAML; Semgrep runs at Layer 2.

FAIL — blocks the repository

- OWASP A08 uses: owner/action with no @ref — an unpinned GitHub Action
- CWE-798 password / secret / token / key / api_key: followed by a literal
- OWASP A05 privileged: true
- OWASP A05 hostNetwork / hostPID / hostIPC: true

WARN — reported, does not block

- CWE-94 {{ request.... / user.... / .env.... }} — possible template injection

Note: Known weak spot: the bare key: rule flags about 7% of real YAML files.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Terraform / HCL

`scan_terraform`

`.tf` `.tfvars` `.hcl`

TOOLS APPLIED

checkov
checkov -f --quiet (--skip-download when offline). Any failed check → FAIL (checkov:N_CIS_IaC_findings). Not installed → WARN.

FAIL — blocks the repository

- CWE-798 password / secret / token = "..."
- OWASP A05 acl = "public-read" (S3)
- OWASP A02 encrypted = false

WARN — reported, does not block

- OWASP A05 cidr_blocks = ["0.0.0.0/0"]

Note: Offline, checkov must be pointed at staged data or it silently contributes nothing.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Dockerfile

`scan_docker`

`Dockerfile*`

TOOLS APPLIED

hadolint
Any lint issue → FAIL (hadolint:dockerfile_issue). Not installed → WARN.

FAIL — blocks the repository

- OWASP A05 a :latest base image
- CWE-494 ADD of an http(s):// URL — no integrity check
- CWE-78 RUN curl | bash or wget | sh
- CWE-798 ENV PASSWORD / SECRET / TOKEN / API_KEY / PRIVATE_KEY baked into a layer

WARN — reported, does not block

- CWE-250 no non-root USER directive

Note: Matched by basename: Dockerfile.dev and Dockerfile.prod count.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

HTML

`scan_html`

`.html` `.htm` `.xhtml`

TOOLS APPLIED

— none —

Inline <script> is ordinary, so every rule targets a construct with no innocent reading. Each was counted on 388 real files.

FAIL — blocks the repository

- eval(); a percent-encoded payload in unescape(); document.write() fed from a decoder
- <script src="http://…">; <base href="http://">; <form action="http://">
- A hidden iframe: display:none, visibility:hidden, zero width or height
- A credential in a name / value attribute pair or a <meta content>
- A cookie read wired to a network sink (matched in both orders)

WARN — reported, does not block

- innerHTML / outerHTML / insertAdjacentHTML
- new Function(; a bare atob(
- A protocol-relative <script src="//…">
- An external meta refresh; a javascript: URI

Note: 2 of 388 real files fail: both genuine eval() calls in the bundled Go trace viewer (vendoring it needs a .transit-allow.json entry). build/, dist/, coverage/ and node_modules/ are pruned first.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

PDF

`scan_pdf`

`.pdf`

TOOLS APPLIED

— stdlib decoder —
Decodes Flate, LZW, ASCIIHex, ASCII85 and RunLength filter chains (64 MB ceiling) and #xx name escapes; two views, so prose cannot fire a key rule.

FAIL — blocks the repository

- CWE-94 /JavaScript, /JS, /OpenAction, /AA — also hex-escaped (/J#61vaScript) and inside compressed object streams
- CWE-78 /Launch
- CWE-506 /EmbeddedFile, or an embedded Windows executable
- Rich media; /SubmitForm, /ImportData
- Known exploit APIs (util.printf, collab.getIcon, media.newPlayer …) and unescape shellcode shapes

WARN — reported, does not block

- Encrypted (/Encrypt) — reported, not decrypted
- A stream behind an undecodable filter, named
- XFA forms; remote or embedded destinations (/GoToR, /GoToE)
- Incremental updates (several %%EOF)

Note: The five PDFs this project ships are the acceptance test: scan them and expect PASS with no finding. Image filters are excluded — they carry pixels, never dictionaries.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Office documents — OOXML (Word, Excel, PowerPoint)

`scan_office`

`.docx` `.docm` `.dotx` `.dotm` `.xlsx` `.xlsm` `.xlsb` `.xltx` `.xltm` `.xlam` `.pptx` `.pptm` `.ppsx` `.ppsm` `.potx` `.potm` `.ppam`

TOOLS APPLIED

stdlib helper

Opens the ZIP in place, never extracts. Reads parts as Office does. XML goes through one linear tokenizer; VBA is decompressed (MS-OVBA) and read. Content is sniffed, whatever the extension says.

FAIL — blocks the repository

- A VBA macro that starts a program or downloads: Shell, WScript.Shell, URLDownloadToFile, PowerShell, kernel32 shellcode calls
- DDE: a DDE / DDEAUTO field (fragments joined across runs), a cmd|'...'!A0 formula, a ddeLink to a shell
- An Excel 4.0 macro sheet that is hidden or calls EXEC / CALL / REGISTER
- Remote fetch: an attachedTemplate over http(s) (template injection); an oleObject / frame / package from a remote host (Follina)
- A link with an ms-msdt:, mhtml:, search-ms:, javascript: or vbscript: scheme; a UNC / smb:// path outside a hyperlink (NTLM leak)
- An INCLUDETEXT / INCLUDEPICTURE / LINK field aimed at a remote host or UNC path; a PowerPoint ppaction://program link
- An embedded executable or script, also under a document name (word/media/image1.png) or inside an OLE Package
- An Equation Editor object carrying a command or URL — also inside an RTF alternate-format chunk
- An external ENTITY; a part named ../...; a PEM key or one of the 18 known token formats in the text

WARN — reported, does not block

- Any VBA macro; one that runs when the document opens; one in a .docx / .xlsx / .pptx
- A visible Excel 4.0 macro sheet; ActiveX; embedded objects that are not readable office documents; an Equation object without a payload
- Content fetched from http(s) when opened; a hyperlink to a file share; a DDE link to an ordinary program
- A bare DOCTYPE; encrypted or unreadable content; a limit reached

Note: Names are compared without their prefix, UTF-16 is decoded, and a part is what the package says it is, wherever stored. Prose is not searched for hostile strings. 217 real and converted documents: no finding. Recall rests on fixtures checked with olevba, olefile and rtfobj. Obfuscated VBA, VBA stomping and encrypted files are not read.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Office documents — legacy OLE2 (.doc .xls .ppt)

`scan_office`

`.doc` `.dot` `.xls` `.xlt` `.xla` `.ppt` `.pot` `.pps` `.ppa`

TOOLS APPLIED

stdlib CFB reader

Reads the compound file's directory and streams (FAT and mini-FAT, bounded), decompresses each VBA module, parses BOUNDSHEET records and OLE Package streams.

FAIL — blocks the repository

- A VBA macro that starts a program or downloads (the module source is decompressed and read)
- A hidden Excel 4.0 macro sheet, or one that runs programs
- An OLE Package holding an executable or script
- An Equation Editor object whose stream carries a command or URL (CVE-2017-11882 family)
- A DDEAUTO field in the document stream

WARN — reported, does not block

- Any VBA macro, or one whose source cannot be read
- An embedded OLE object; an Equation object without a payload
- Encrypted content; a file that cannot be parsed

Note: .dot is Graphviz, .pot is gettext and a .doc is sometimes plain text: the content is sniffed, and text goes back to the unknown-file path (PASS). A password-protected file is reported, not opened.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

OpenDocument (LibreOffice, OpenOffice)

`scan_office`

`.odt` `.ods` `.odp` `.odg` `.ott` `.ots` `.otp` `.otg` `.fodt` `.fods` `.fodp` `.fodg`

TOOLS APPLIED

stdlib helper

Opens the ZIP (or reads the flat XML). Looks at content.xml, styles.xml, the manifest, Basic/ and Scripts/. meta.xml is not searched for links.

FAIL — blocks the repository

- A macro bound to office:load / office:new / office:create / view-created that lives IN the document
- A StarBasic, Python or BeanShell script calling Shell, os.system, subprocess, sockets, urllib, ctypes …
- curl | sh, encoded PowerShell or certutil -urlcache in an embedded script
- A DDE link to a shell; a UNC path outside a hyperlink; an executable stored in the package
- An external ENTITY; a PEM key or a known token format in the text

WARN — reported, does not block

- Any embedded macro, or a hyperlink that runs one on click
- An image or object linked from http(s) or a file path; a DDE link to an ordinary program
- Encrypted parts; a limit reached

Note: Calibrated on 123 real templates. Not findings: every file declares the script: namespace (a rule on it fails 100%); 10 of 123 bind an event to a macro the APPLICATION ships (application:…); meta:template links to the source template.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Rich Text Format

`scan_office`

`.rtf`

TOOLS APPLIED

stdlib helper

RTF has no container: objects are hex inside the text, and Word reads them through junk. Object groups are decoded and each OLE 1.0 object is opened.

FAIL — blocks the repository

- An Equation.3 object whose data carries a command or URL (CVE-2017-11882 family)
- A linked object with \objautlink and \objupdate that fetches from a remote source
- A {*\template …} over http(s) or a UNC path
- A Package object holding an executable or script
- A DDEAUTO field

WARN — reported, does not block

- An embedded OLE object (not an exploit shape)
- An unterminated object group or a limit reached

Note: The hex is read through line breaks and empty groups, as Word reads it. Group scanning is bounded: an unterminated group at every object would otherwise cost the whole file each.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

SQL

`scan_sql`

`.sql`

TOOLS APPLIED

— none —

Four built-in rules.

FAIL — blocks the repository

- CWE-78 xp_cmdshell, sp_OACreate, OPENROWSET, BULK INSERT
- CWE-89 DROP TABLE / DATABASE / SCHEMA / VIEW / FUNCTION / PROCEDURE
- CWE-89 injection patterns: ; --, '--, OR 1=1, AND 1=1
- CWE-89 UNION [ALL] SELECT

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: A migration that legitimately drops a table fails: use .transit-allow.json.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

JSON

`classify_file (inline rule)`

`.json`

TOOLS APPLIED

— none —

One built-in rule. package.json and composer.json are routed to the manifest scanner by name.

FAIL — blocks the repository

- CWE-798 "password" / "secret" / "token" / "api_key" / "private_key": "<4+ character literal>"

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: After the bracket-expression fix this rule sees values containing an "s": 12 of 2252 real JSON files flag (9 before).

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Documentation and text

`classify_file (inline rule)`

`.md` `.txt` `.rst`

TOOLS APPLIED

— none —

One built-in rule. CMakeLists.txt and requirements*.txt end in .txt but are routed by name first.

FAIL — blocks the repository

- CWE-798 password / secret / api_key followed by : or = and a literal value

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: A README that quotes an attack string (a credential or a traversal example) fails: INSTALL.md and CLAUDE.md of this project do. That is what .transit-allow.json is for.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Native binaries

`scan_binary`

`.so` `.dll` `.dylib` `.exe` `.elf` `.bin`

TOOLS APPLIED

strings

Extracts readable strings and looks for http://, https://, /etc/passwd, /bin/sh, exec, shell, reverse, c2server.

FAIL — blocks the repository

- binary:unexpected_binary_in_AI_repo — EVERY file of these types, whatever it contains
- binary:IOC_strings_detected — in addition, when the strings search hits

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: .bin is blocked, not analysed: a harmless weights file fails exactly like an ELF, while the same pickle named .pkl or .pt passes (see Unknown extensions). Content-based routing would fix both.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Archives

`scan_archive`

`.zip` `.tar` `.gz` `.tgz` `.bz2` `.xz`

TOOLS APPLIED

unzip -l
.zip only: lists the entries and looks for '..' in a path.

FAIL — blocks the repository

- archive:requires_dedicated_rescan — EVERY archive
- CWE-22 a .zip entry whose path contains '..' (zip-slip)

WARN — reported, does not block

No reviewable tier: every rule here blocks.

Note: An archive is not opened: it must be extracted in an isolated environment and rescanned. Java archives are the exception (next slide).

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Java archives

`scan_jar`

`.jar` `.war` `.ear` `.aar`

TOOLS APPLIED

stdlib zipfile helper

Opened in place, never extracted. Reads entries like the JVM does; bounded by entry, byte, text, size, time and depth limits — each one reached is reported.

FAIL — blocks the repository

- Zip-slip: a path with .., a leading / or a drive letter; a symlink out of the archive; a local record the central directory does not list
- A .class that is not bytecode; a PE / ELF / Mach-O under a non-native name or glued in front of the archive
- A manifest Class-Path naming an http(s):// or ftp:// jar
- curl | sh, a reverse shell, encoded PowerShell, certutil -urlcache — in a resource, UTF-16 text, a class constant or attribute
- A PEM private key at any line width; any of the 18 known credential formats
- gradle-wrapper.jar / maven-wrapper.jar with entries outside the expected tree or no main class

WARN — reported, does not block

- EVERY jar: its bytecode is not reviewed, so a clean jar still warns
- Java agents; native libraries; scripts; nested archives; symlinks; duplicate names
- Encrypted or unreadable entries; any limit reached (inspection_incomplete)
- Every wrapper jar: verify its SHA-256 yourself

Note: 365 real jars: one FAIL, a genuine embedded test key. .war / .ear / .aar are routed but unmeasured. Bytecode is not decompiled. A jar under build/, dist/, target/ or vendor/ is pruned and never scanned.

Also applied to every file: L1 gitleaks · detect-secrets · ClamAV · YARA | L2 Semgrep | L4 universal patterns (credentials, key material, traversal, SSRF, weak crypto) | L6 ScanCode

Unknown extensions — what is allowed

Anything not routed above. It is not skipped: it still passes every layer except Layer 5.

HOW IT IS HANDLED

`#!…python`	→ **scan_python**
`#!…/bash, /sh, /zsh, /ksh, env bash|sh`	→ **scan_shell**
`#!…ruby`	→ **scan_ruby**
`#!…node`	→ **scan_javascript**
`#!…php`	→ **scan_php**
`#!…pwsh, powershell`	→ **scan_powershell**

- No shebang: file --mime-type. x-executable, x-sharedlib or x-dosexec → FAIL (binary:unexpected_executable_mime).
- Any other type → PASS. That is "allowed".
- If the file command is not installed the MIME check is skipped and the file passes.

STILL APPLIED

- L1 gitleaks, detect-secrets, ClamAV and YARA — whole directory
- L2 Semgrep rulesets — whole directory
- L4 universal patterns on this very file: hardcoded credentials, the 18 known token formats, PEM private keys, path traversal in three contexts, SSRF, weak crypto and PRNG, debug artefacts, logging suppression
- L6 ScanCode licences and CVEs
- Examples that land here: .png .jpg .csv .toml .ini .cfg .conf .lock .env .svg .proto .safetensors

WHAT PASSES UNEXAMINED

- Source in a language with no scanner: Perl, Lua, VBScript, Groovy (outside Gradle), Scala, Swift, Dart, Elixir … — no per-language rule runs, only Layer 4.
- Pickled models and weights: .pkl .pickle .pt .pth .ckpt .joblib. A pickle that runs os.system PASSED when tested; loading one executes it.
- A .bin pickle is the opposite case: blocked with every other .bin, harmlessly or not.
- Content disguised by name: a script saved as .txt gets only the document rule.
- Remedy for a legitimate file: .transit-allow.json (FAIL → WARN, with a reason) or .transitignore.

Index — every routed extension and basename

Extension → slide number.

*.gradle	p. 20	.exe	p. 35	.odg	p. 30	.psd1	p. 16	.xlsm	p. 28
*.gradle.kts	p. 20	.fodg	p. 30	.odp	p. 30	.psm1	p. 16	.xlsx	p. 28
.aar	p. 37	.fodp	p. 30	.ods	p. 30	.psrc	p. 16	.xlt	p. 29
.bash	p. 15	.fods	p. 30	.odt	p. 30	.pssc	p. 16	.xltm	p. 28
.bat	p. 17	.fodt	p. 30	.otg	p. 30	.py	p. 4	.xltx	p. 28
.bin	p. 35	.go	p. 10	.otp	p. 30	.rb	p. 9	.xml	p. 22
.bz2	p. 36	.gz	p. 36	.ots	p. 30	.rs	p. 12	.xz	p. 36
.c	p. 11	.h	p. 11	.ott	p. 30	.rst	p. 34	.yaml	p. 23
.cc	p. 11	.hcl	p. 24	.pdf	p. 27	.rtf	p. 31	.yml	p. 23
.cdxml	p. 16	.hpp	p. 11	.php	p. 8	.sh	p. 15	.zip	p. 36
.cjs	p. 6	.htm	p. 26	.pom	p. 22	.so	p. 35	.zsh	p. 15
.cmake	p. 19	.html	p. 26	.pot	p. 29	.sql	p. 32	CMakeLists.*.txt	p. 19
.cmd	p. 17	.jar	p. 37	.potm	p. 28	.tar	p. 36	CMakeLists.txt	p. 19
.cpp	p. 11	.java	p. 7	.potx	p. 28	.tf	p. 24	composer.json	p. 21
.cs	p. 14	.js	p. 6	.ppa	p. 29	.tfvars	p. 24	conanfile.py	p. 5
.dll	p. 35	.json	p. 33	.ppam	p. 28	.tgz	p. 36	constraints*.txt	p. 21
.doc	p. 29	.jsx	p. 6	.pps	p. 29	.ts	p. 6	Dockerfile*	p. 25
.docm	p. 28	.ksh	p. 15	.ppsm	p. 28	.tsx	p. 6	GNUmakefile*	p. 18
.docx	p. 28	.kt	p. 13	.ppsx	p. 28	.txt	p. 34	gradle-wrapper.properties	p. 20
.dot	p. 29	.kts	p. 13	.ppt	p. 29	.war	p. 37	gradle.properties	p. 20
.dotm	p. 28	.mak	p. 18	.pptm	p. 28	.xhtml	p. 26	Makefile*	p. 18
.dotx	p. 28	.make	p. 18	.pptx	p. 28	.xla	p. 29	makefile*	p. 18
.dylib	p. 35	.md	p. 34	.ps1	p. 16	.xlam	p. 28	package.json	p. 21
.ear	p. 37	.mjs	p. 6	.ps1xml	p. 16	.xls	p. 29	requirements*.txt	p. 21
.elf	p. 35	.mk	p. 18	.psc1	p. 16	.xlsb	p. 28	setup.py	p. 5

Unknown extensions: p. 38